

PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionen» mit booleschen Rückgabewerten: $P, Q(n), R(x, y, z)$.

$A \Rightarrow B \Leftrightarrow (\neg A \vee B)$

$\neg(A \wedge B) \Leftrightarrow (\neg A \vee \neg B)$

$\neg(\neg A) \Leftrightarrow A$

$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$

$A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$

$A \wedge (A \wedge B) \Leftrightarrow A$

$A \vee (A \vee B) \Leftrightarrow A$

$\neg(A \wedge B) \Leftrightarrow (\neg A \vee \neg B)$

$\neg(A \vee B) \Leftrightarrow (\neg A \wedge \neg B)$

Begriff

Bedeutung

Abtrennungsregel

$(A \wedge (A \Rightarrow B)) \Rightarrow B$

Kommutativität

$(A \wedge B) \Leftrightarrow (B \wedge A)$
 $(A \vee B) \Leftrightarrow (B \vee A)$

Assoziativität

$A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$
 $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$

Distributivität

$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$
 $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$

Absorption

$A \vee (A \wedge B) \Leftrightarrow A$
 $A \wedge (A \vee B) \Leftrightarrow A$

Idempotenz

$A \vee A = A$
 $A \wedge A = A$

Doppelte Negation

$\neg(\neg A) \Leftrightarrow A$

Konstanten

$W = \text{wahr}$
 $F = \text{falsch}$

de Morgan

$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$
 $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$

NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$
$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

SPRACHEN

Regulär

DEA

NEA

Regex

Kontextfrei

CFG

PDA

Palindrome

$\{0^n 1^n \mid n \geq 0\}$

Turing-erkennbar

Primzahlen

$\{a^n b^n c^n \mid n \geq 0\}$

Alle Sprachen

Überabzählbar

Q unendlich

$F \subset Q$

$A = (Q, \Sigma, \delta, q_0, F)$

Zustände:

$\{q_0, q_1, \dots, q_n\}$

Übergangsfunktion:

$\delta: Q \times \Sigma \rightarrow Q$

Startzustand:

$q_0 \in Q$

Akzeptierzustände:

$F \subset Q$

Σ

Start δ

q_0

0

Beispiel 5

Start q_0

q_0

q_1

q_2

q_3

q_0

q_1

q_2

q_3

q_0

q_1

q_2

q_3

q_0

q_1

q_2

q_3

Algorithmus

1) $q \in F, q' \notin F$

2) Unterscheidbar nach Übergang

3) Iteration bis keine Änderung mehr

$\Rightarrow q_1$ und q_2 sind nicht unterscheidbar

TODO:

reduce confusion

x

$y 0$

$z 1$

x

$y 0$

$y 0$

$z 1$

5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 1 bleibt

6) $xy^k z \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition

$A = (Q, \Sigma, \delta, q_0, F)$

– Endliche Menge von Zuständen: Q

– Alphabet: Σ

– Übergangsfunktion: $\delta: Q \times \Sigma \rightarrow P(Q)$

– Startzustand: $q_0 \in Q$

– Akzeptierzustände: $F \subset Q, 0,1$

Akzeptieren

Ein NEA A **akzeptiert** das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

– Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.

– Erlaube Alternativen

Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

WÖRTER EINER REGULÄREN SPRACHE

ÜBERGANGSFUNKTIONEN FÜR WÖRTER

$\delta: Q \times \Sigma^* \rightarrow Q: (q, w = a_1, \dots, a_n) \mapsto \delta(\dots \delta(q, a_1), a_2), \dots, a_n)$

Übergänge

ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

WORT AKZEPTIEREN

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

AKZEPTIERTE SPRACHE, REGULÄRE SPRACHE

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$

Die Sprache $L \subset \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3: DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist eine ungerade Zahl im Dreiersystem}\}$ akzeptiert

Start q_0

$0,2$

1

$0,2$

g

u

g

u

g

u

MYHILL-NERODE AUTOMAT

Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww'\}$. Insbesondere $L(\varepsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:

- $\Sigma = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\sigma) = L$
- $F = \{L(w) \in Q \mid \varepsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4: $\Sigma = \{0, 1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

Start q_0

1

w

0

1

$L(w)$

$L(\varepsilon) = L$

$L(0) = \{w \in \Sigma^* \mid |w|_0 \text{ ungerade}\}$

$L(1) = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$

q_0

q_1

q_0

q_1

Beispiel 6: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

1) Annahme: L ist regulär

2) $\exists N \in \mathbb{N}$, Pumping Length

3) $w = 0^N 1^N$

4) Unterteilung $w = xyz$

NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$\neg(P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg(P_1) \vee \dots \vee \neg(P_n) \Leftrightarrow \exists i \in \{1, \dots, n\} (\neg P_i)$

MENGEN

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

OPERATIONEN

Begriff

Bedeutung

Vereinigung

$A \cup B = \{x \mid x \in A \vee x \in B\}$

Schnittmenge

$A \cap B = \{x \mid x \in A \wedge x \in B\}$

Komplement

$\bar{A} = \{x \mid x \notin A\}$

Differenz

$A \setminus B = \{x \in A \mid x \notin B\}$

TUPEL

$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$

n -Tupel:
$$\prod_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik

Anwendungsfall

Konstruktiver Beweis

Liefert explizite «Lösung» / Algorithmus

Widerspruchsbeweis

Geeignet für Unmöglichkeitss Aussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$

Vollständige Induktion

Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

Beispiel 1: Wortlänge

$|\varepsilon| = 0$

$|01010|_1 = 2$

$|a^3 b^5| = 8$

$|01010|_0 = 3$

$|01010|_3 = |01291|^7| = 7|1291| = 7 \cdot 4 = 28$

Beispiel 2: Sprache

$L = \Sigma^* \subset \Sigma^*$

$L = \emptyset \subset \Sigma^*$, die leere Sprache

$\Sigma = \{0, 1\}$, Sprache aller Binärstrings: $L = \Sigma^*$

$\Sigma = \text{Unicode}, J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$

$M = \text{«Maschine»}, L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$

Beobachtungen 1

1.1. $|w^n| = n \cdot |w|$

1.2. Zu jeder Maschine M gibt es eine Sprache $L(M)$

DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition

Tabellenform

Zustandsdiagramm von A

Beispiel 7: $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei } 0\}$

DEA-Lösung

NEA-Lösung

Beispiel 8: $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

1) Annahme: L ist regulär

2) $\exists N \in \mathbb{N}$, Pumping Length

3) $w = 0^N 1^N$

4) Unterteilung $w = xyz$

AutoSpr | FS26

Seite 1

4) Startvariable: S

Es ist nicht garantiert, dass es für die Ableitung eines Wortes nur einen einzigen Syntaxbaum gibt. Man sagt, die Grammatik ist **nicht eindeutig**, wenn sie mehrere Syntaxbäume für die gleichen Wörter erlaubt.

REGULÄRE OPERATIONEN
Die Klasse der kontextfreien Sprachen ist abgeschlossen unter regulären Operationen.

Reguläre Sprachen sind kontextfrei.

Grammatik für reguläre Operationen

Seien L_1 und L_2 kontextfreie Sprachen mit Grammatiken $G_i = (V_i, \Sigma, R_i, S_i)$. Wir nehmen an, dass die Variablen- und Regelmengen disjunkt sind, also $V_1 \cap V_2 = \emptyset \wedge R_1 \cap R_2 = \emptyset$. Zudem sei S_0 eine Variable, die nicht in $V_1 \cup V_2$ enthalten ist. Dann gilt:

Alternative $L_1 \mid L_2$ ist kontextfrei mit der Grammatik

$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 \mid S_2\}, S_0)$

Verkettung $L_1 L_2$ ist kontextfrei mit der Grammatik

$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 S_2\}, S_0)$

Operation L_1^ ist kontextfrei mit der Grammatik

$G = (V_1 \cup \{S_0\}, \Sigma, R = R_1 \cup \{S_0 \rightarrow S_0 S_1, S_0 \rightarrow \varepsilon\}, S_0)$

Kontextfrei

Regeln ohne Kontext

Es spielt keine Rolle, in welchem Kontext das Zeichen A vorkommt, die Regel kann immer angewendet werden

$S \rightarrow A \mid C$
 $A \rightarrow a$
 $A \rightarrow aAb$
 $C \rightarrow bA$

Regeln mit Kontext

Je nach **Kontext** kann eine Regel nicht unbedingt angewendet werden

$S \rightarrow C \mid aC \mid bC$
 $aC \rightarrow A$
 $bC \rightarrow B$
 $S \rightarrow aC \rightarrow A$
 $S \rightarrow bC \rightarrow B$
 $S \rightarrow C \rightarrow ?$

Chomsky-Normalform (CNF)

Eine CFG ist in **Chomsky-Normalform**, wenn S auf der rechten Seite nicht vorkommt und jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$ ist, zusätzlich ist die Regel $S \rightarrow \varepsilon$ erlaubt.

Umwandlung

- 1) Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)
- 2) ε -Regeln: $A \xrightarrow{\varepsilon} B \Rightarrow A$ kann weggelassen werden $\Rightarrow \begin{cases} B \rightarrow AC \\ A \rightarrow C \end{cases}$
- 3) Unit-Rules: $A \xrightarrow{\varepsilon} B \Rightarrow A$ kann man wie aus B auch CD machen $\Rightarrow \begin{cases} A \rightarrow CD \\ B \rightarrow CD \end{cases}$
- 4) Verkettungen: $A \rightarrow u_1 u_2 \dots u_n$ ersetzen durch $A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$ und falls u_i ein Terminalsymbol ist: $A_{i-1} \rightarrow U_i A_i, U_i \rightarrow u_i$.

Folgerungen

- 1) Ableitung eines Wortes $w \in L(G)$ ist immer in $2|w| - 1$ Regelanwendungen möglich. Beweis:
 - $|w| - 1$ Regeln der Form $A \rightarrow BC$ um aus S ein Wort aus $|w|$ Variablen zu erzeugen
 - $|w|$ Regeln der Form $A \rightarrow a$ um das Wort w zu erzeugen
 - $\Rightarrow$ insgesamt $2|w| - 1$ Regelanwendungen
- 2) Deterministischer Parse-Algorithmus mit Laufzeit $O(n^3) \Rightarrow AB \mid CD \mid CU \mid SSB \rightarrow 1$

$S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \varepsilon$

Beispiel 9:

1) Startvariable

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b \mid \varepsilon$

2) ε -Regel

$S_0 \rightarrow S$
 $S \rightarrow ASA \mid aB \mid a$
 $A \rightarrow B \mid \varepsilon \mid S$
 $B \rightarrow b$
 $S_0 \rightarrow S$
 $S \rightarrow ASA \mid SA \mid AS \mid aB$
 $A \rightarrow B \mid S$
 $B \rightarrow b$

3) Unit-Regel

$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid S_0$
 $S \rightarrow ASA \mid SA \mid AS \mid aB \mid S$
 $A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid ab \mid AS \mid SA \mid AS \mid aB \mid a$
 $B \rightarrow b$

4) Komplexe Regeln

$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$
 $A_1 \rightarrow SA$
 $U \rightarrow a$
 $B \rightarrow b$

Parsing

Cocke-Younger-Kasami Algorithmus (CYK)

Gegeben

- 1) Grammatik $G = (V, \Sigma, R, S)$
- 2) Variable $A \in V$
- 3) Wort $w \in \Sigma^*$

Frage

Ist w ableitbar? Formell geschrieben: $A \xrightarrow{*} w$
- Spezialfall $w = \varepsilon$: $A \xrightarrow{*} \varepsilon \Leftrightarrow A \rightarrow \varepsilon \in R$ (Epsilonregel, $A = S$)
- Spezialfall $|w| = 1$: $A \xrightarrow{*} w \Leftrightarrow A \rightarrow w \in R$ (Terminalsymbolregel)
- Fall $|w| > 1$:

$A \xrightarrow{*} w \Rightarrow \exists \begin{cases} A \rightarrow BC \in R \\ w = w_1 w_2 \\ w_i \in \Sigma^* \end{cases} \text{ mit } \begin{cases} B \xrightarrow{*} w_1 \\ C \xrightarrow{*} w_2 \end{cases}$

Ableitungsdreieck

Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle füllen

Prinzip

- Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
- Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

Beispiel

Parse des Wortes $()()()$

$S \rightarrow SD$
 $A \rightarrow ($
 $C \rightarrow [$
 $D \rightarrow]$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heißen **korrespondierende Felder** im Ab-

leitungsdreieck des Feldes (k, l) .

TODO:

rekursion vs iteration (buch 149)

Backus-Naur-Form (BNF)

Spezifikation der Regeln in maschinen-lesbarer Form:

- Variablen: **<variablen-name>**
- Einzelne Zeichen: **A**
- Zeichenketten: **'BEISPIEL'**
- Regeln: **<variablen-name> ::= Ausdruck**
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch $|$

Beispiel 10: BNF für Expression-Term-Factor Grammatik

Ausgangsgrammatik

$\text{expression} \rightarrow \text{expression} + \text{term} \mid \text{term}$
 $\text{term} \rightarrow \text{term} * \text{factor} \mid \text{factor}$
 $\text{factor} \rightarrow (\text{expression}) \mid N$
 $N \rightarrow NZ \mid Z$
 $Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Backus-Naur-Form

<expression> ::= <expression> + <term> |
<term> ::= <term> * <factor> |
<factor> ::= (<expression>) | <number>
<number> ::= <number> <digit> | <digit>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Extended Backus-Naur-Form (EBNF)

Definition

- Literale in Anführungszeichen oder Apostroph
- = statt ::=
- Variablen ohne < >, dürfen Leerzeichen enthalten
- Komma für Verkettung
- Regel-Endzeichen ;
- Kommentare (* ... *)
- Optionale Wiederholung { ... }
- Option [...]
- Gruppierung (...)
- Ausnahme: -

Achtung!

- Keine Operatorpriorisierung
- Zweideutig!
- Keine eindeutige Evaluation!

Beispiel 11: EBNF für Expression-Term-Factor Grammatik

expression = expression, "+", term | term, "f", "f" $\rightarrow$ \$
term = term, "*", factor, | factor ;
factor = "(", expression, ")" | number ;
number = digit, { digit } ;
digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

Rechtsrekursion

Expression-Term-Factor-Grammatik verwendet Links-Rekursion $\Rightarrow$ korrekte Auswertung von links nach rechts. Parsetree Wert aber Ausdrücke von rechts nach links aus! Alternative Expression-Term-Factor definition mit Rechtsrekursion statt Linksrekursion:

$\text{expression} \rightarrow \text{term expression}'$
 $\text{expression}' \rightarrow + \text{term expression}' \mid \varepsilon$
 $\text{term} \rightarrow \text{factor term}'$
 $\text{term}' \rightarrow * \text{factor term}' \mid \varepsilon$
 $\text{factor} \rightarrow (\text{expression}) \mid N$
 $N \rightarrow NZ \mid Z$

$Z \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

TODO:

grid $\rightarrow$ table

Stack

Unendlich grosser Speicher

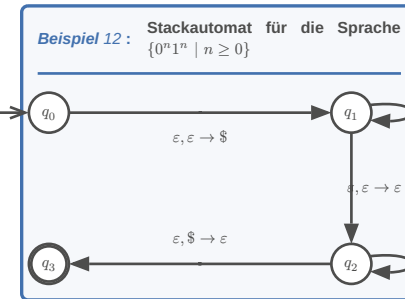
- Immer nur das oberste Element sichtbar
- Beliebige tief

Stackautomat / Pushdown Automaton (PDA)

Kann Sprachen akzeptieren, die nicht regulär sind, ist aber nicht deterministisch. **Beachte:** $\Gamma \neq \Sigma$ möglich

Definition

- Stackautomat $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$
- 1) Q : Zustände
 - 2) Σ : Eingabe-Alphabet
 - 3) Γ : Stack-Alphabet
 - 4) $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow P(Q \times \Gamma_\varepsilon)$
 - 5) $q_0 \in Q$: Startzustand
 - 6) $F \subset Q$: Akzeptierzustände
- a vom Input
 b vom Stack entfernen (Bedingung)
 c auf den Stack



TODO:

Book 167-168, shift/reduce

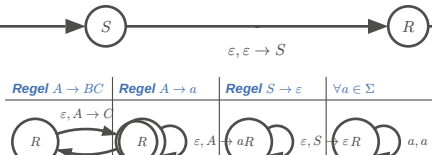
TODO:

diagrams? (slides 8/12)

Ableitung aus CNF

Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P , der L akzeptiert, $L = L(P)$.

Grundgerüst



PDA -> CFG

Variablen

Wörter beschreiben Pfade durch den Automaten: Variable A_{pq} = Wörter, die von p nach q führen mit leerem Stack

Regeln

Regeln beschreiben, wie sich Wege zerlegen lassen: $A_{pq} \rightarrow A_{pr} A_{rq}$ = Wege von p nach q können verstanden werden als Wege von p nach r und von dort nach q

Stackautomat standardisieren

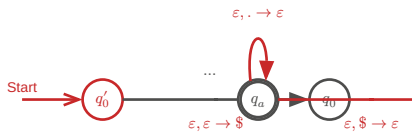
TODO:

better diagrams, book 181

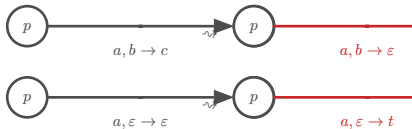
- 1) Nur ein Akzeptierzustand: Neuen Akzeptierzustand q_a und Übergänge von allen vorherigen Endzuständen zu q_a erstellen.



- 2) Stack leeren: Mit einem Element $\$$ erweitern, sodass garantiert werden kann, dass der Stack am schluss leer ist.



- 3) Jeder Übergang legt entweder ein Zeichen auf den Stack oder entfernt eines



TODO:

Book 182,183

Beispiel 13

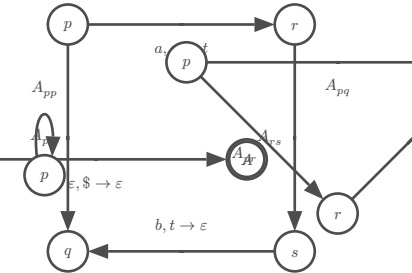
TODO:

visualization

Grammatik ablesen

Ausgangspunkt: standardisierter PDA mit Startzustand q_0 und $F = \{q_a\}$.

- 1) Startvariable: A_{q_0, q_a}
- 2) Regeln:



Beispiel 14

TODO:

Nicht Kontextfreie Sprachen

Pumping Lemma

Pumping Lemma für CFL

Ist L eine CFL, dann gibt es eine Zahl N , die die Pumping Length, derart, dass jedes Wort $w \in L$ mit $|w| \geq N$ zerlegt werden kann in fünf Teile $w = uvxyz$ derart, dass

- 5) Bewegung immer nur eine Zelle nach links oder rechts
- 6) Kein weiterer Speicher (nur die Zustände eines endlichen Automaten und das eine Zeichen)

Links	Rechts
$a \rightarrow b, l, L$ $\dots a_1 a_2 p \ a \ a_4 a_5 \dots$ $\dots a_1 \ q \ a_2 \ b \ a_4 a_5 \dots$	$a \rightarrow b, l, L$ $\dots a_1 a_2 p \ q \ a_4 a_5 \dots$ $\dots a_1 a_2 \ b \ q \ a_4 a_5 \dots$

Turing-erkennbare Sprache: L heisst **Turing-erkennbar**, wenn es eine TM M gibt mit $L = L(M)$.

Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.

Begriff	Bedeutung
Record	Speichereinheit fester grösse
Bandalphabet	Alphabet Γ , welches aus Speicherzellen (Stacks) besteht
Schreib-/Lesekopf	Aktuelle Schreib-/Leseposition

- ### Definition
- (deterministische) Turing-Maschine
- $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$
- Q : **Zustände**
 - Σ : Alphabet
 - Γ : Bandalphabet, $\cup \in \Gamma \times \Sigma$
 - δ : $Q \times \Gamma \rightarrow Q \times \Gamma \times \Sigma$
 - $\{L, R\}$
 - $q_0 \in Q$: Startzustand
 - $q_{\text{accept}} \in Q$: Akzeptierzustand
 - $q_{\text{reject}} \in Q$: Ablehnzustand, $q_{\text{accept}} \neq q_{\text{reject}}$
- ### Zustandsdiagramm
- $\delta(p, a) = (q, b, L) : \text{ } \textcircled{p}$
- ### Übergänge
- Übergang möglich, wenn a unter dem Schreib- / Lesekopf
 - Aktuelles Feld auf dem Band wird mit b überschrieben
 - Kopfbewegung: L links, R rechts

$$w \in L(G) \Rightarrow S \stackrel{*}{\Rightarrow} w$$
$$\begin{aligned} A &\stackrel{*}{\Rightarrow} vxy \\ A &\stackrel{*}{\Rightarrow} x \end{aligned}$$

TODO:
diagram

Begriff	Bedeutung
Endlich	Eine Menge A heisst endlich , wenn jede Inklusion $A \rightarrow A$ auch eine Bijektion ist
Abzählbar unendlich	Eine Menge A heisst abzählbar unendlich , wenn es eine Bijektion $\mathbb{N} \rightarrow A$ gibt, also $A \approx \mathbb{N}$. Bsp: $\mathbb{R}, \mathbb{Z}, \mathbb{Q}, \Sigma^*$, Menge aller DEAs/NEAs/PDAs/CFGs
Überabzählbar unendlich	Eine Menge A heisst überabzählbar unendlich , wenn sie nicht abzählbar unendlich ist. Bsp: $\mathbb{C}, P(\mathbb{N})$, Menge aller Sprachen $P(\Sigma^*)$
Gleich mächtig	Mengen A und B heissen gleich mächtig , $A \approx B$, wenn es eine Bijektion $A \rightarrow B$ gibt
Unendlich	Eine Menge A heisst unendlich , wenn sie gleich mächtig wie eine Teilmenge ist

Beispiel 16 : $\{0^n 1^n \mid n \geq 0\}$

Beispiel 17: $L = \{0^{n+1} \mid n \geq 0\}, w = 0011$

The diagram shows a 32-bit register. The top 4 bits are labeled 'ALU result' and contain the value 1. The bottom 28 bits are labeled 'data' and contain the value 0x418737. The register is divided into four sections: the first 4 bits (ALU result), the next 4 bits (containing 1), the next 16 bits (containing 0x41), and the last 8 bits (containing 0x8737). The value 0x418737 is shown in the top right corner.

TODO:
buch ..250, slides 14

Um eine Mehrbandmaschine mit n Bändern auf einer einfacheren Maschine zu simulieren, kann die unabhängige Bewegung der Schreib-/Leseköpfe mithilfe eines $2n$ -spurigen Bandes (Daten + Zeiger für Schreib-/Lesekopf) nachgebildet werden.

Simulierbar in Zeit $O(t(n))$

Harvard- und von Neumann-Architektur

Die AVR-Mikrokontroller-Familie von Atmel verwendet intern drei unabhängige Datenpfade:

- Flash-Speicher, der den Programmcode enthält
- Statisches RAM, welches zur Laufzeit veränderliche Daten speichert
- Peripheriegeräte.

Die von Neumann-Architektur vereinheitlicht Daten- und Programmspeicher.

EINSEITIG UNENDLICHES BAND

q_0 diagram (buch 235)

Eine TM erkennt das Wort $w \in \Sigma^*$, wenn die Maschine auf dem Input w im Zustand $q_{\text{akzeptiert}}$ anhält.

Sind L_1 und L_2 Turing-erkennbare Sprachen, dann sind auch der Durchschnitt $L_1 \cap L_2$ und die Vereinigungsmenge $L_1 \cup L_2$ Turing-erkennbar.

Es gibt überabzählbare viele Sprachen $L \subset \Sigma^*$, die **nicht** Turing-erkennbar sind.

AUFZÄHLER
Aufzähler: TM mit einem Drucker, mit dem Wörter ausgedruckt werden können.

L ist **rekursiv aufzählbar** wenn es einen Aufzähler gibt, der alle Wörter aufzählen kann. L ist dann auch Turing-erkennbar.

ENTSCHEIDER UND ENTSCHEIDBARE SPRACHEN

Deterministisch: Eine TM M heisst **Entscheider**, wenn sie auf jedem Input w anhält.

nichtdeterministisch. Eine nichtdeterministische TM M heisst **Entscheider**, wenn jede Berechnungsgeschichte terminiert.

NICHTDETERMINISTISCHE TM

Bei jedem Übergang maximal TM
 N verschiedene Möglichkeiten: Verwende 3 Bänder:

en: $\delta : Q \times \Gamma \rightarrow P(Q \times \Gamma \times L, R\}$)

$\Rightarrow$ Mehrere mögliche Berechnungswege

- 1) Arbeitsband
- 2) Kopie von w
- 3) Liste aller Folgen von Wahlmöglichkeiten

$v \in L(M)$ wenn es einen Berechnungsweg gibt, der zu

accept limit. (auc